

TEAM PARADOX

Road Damage Detection using YOLOv8

Crackathon IIT Bombay 2025

Team: Team_Paradox

Model architecture:

- Uses **CSPDarknet backbone**
- PAN-FPN neck
- Anchor-free detection head

We used the YOLOv8s architecture from *Ultralytics*

Include:

- Dataset provided by Crackathon
- YOLO-format labels
- 5 classes

0 – Longitudinal crack

1 – Transverse crack

2 – Alligator crack

3 – Other corruption

4 – Pothole

Data Augmentation & Training Strategy (½–1 page)

What YOLO already applies (IMPORTANT):

YOLOv8 automatically applies:

- Mosaic augmentation
- Random scaling
- Horizontal flip
- HSV color augmentation

Data augmentation was handled internally by YOLOv8, including mosaic augmentation, random scaling, and color jittering, which improved robustness to lighting variations and object scale diversity.

Training details:

- Image size: 768
- Optimizer: AdamW (default)
- Loss: CloU + classification + objectness
- Device: CPU (hardware constrained)

Hyperparameter Tuning Experiments

Multiple experiments were conducted by varying input image resolution and confidence thresholds. Increasing image size from 640 to 768 improved localization accuracy for fine cracks. Confidence thresholds between 0.2–0.3 were tested to balance recall and precision.

Techniques Used to Improve Baseline:-

- Higher resolution (768)
- Threshold tuning
- Class-wise evaluation
- Multiple training runs
- Best-weight selection

Compared to the baseline YOLOv8s configuration, performance improvements were achieved by increasing input resolution, tuning confidence and IoU thresholds, and selecting the best-performing checkpoint based on validation mAP.

mAP@50 $\approx$ 58%

mAP@50–95 $\approx$ 30%

While higher-capacity models such as YOLOv8m can achieve superior performance on high-end GPUs, our approach demonstrates competitive results under limited hardware constraints.

Conclusion

This project demonstrates the effectiveness of YOLOv8 for automated road damage detection. Despite hardware limitations, meaningful performance was achieved through systematic tuning and data-driven experimentation.

CODE USED :-

To import ultralytics:-

```
from ultralytics import YOLO  
  
model = YOLO(...)
```

To active yolo eve:

```
(yolo_env) C:\crackathon>
```

To generate predictions:-

```
python - <<EOF  
  
from ultralytics import YOLO  
  
model = YOLO("runs/detect/train/weights/best.pt")  
  
model.predict(
```

```
source="../paradox/randomized_dataset/test/images",
conf=0.20,
save_txt=True,
save=False,
project="runs/detect",
name="predict_conf20"
)
EOF
```

For high recall:-

```
python - <<EOF
from ultralytics import YOLO

model = YOLO("runs/detect/train/weights/best.pt")

model.predict(
    source="../paradox/randomized_dataset/test/images",
    conf=0.20,
    save_txt=True,
    save=False,
    project="runs/detect",
```

```
name="predict_conf20"
```

```
)
```

```
EOF
```

To zip the TXT files:-

```
runs\detect\predict_conf25\labels
```